TRIBHUVAN UNIVERSITY

Institute of Engineering

Pulchowk Campus

A Project REPORT ON

GENETIC ALGORITHMS

Submitted By:

Pratik Ghimire : 080BEI029

Rakesh Yadav : 080BEI031

Saurab Gyawali : 080BEI039

Submitted To:

Department of Electronics and Computer Engineering

ABSTRACT

Feature selection constitutes a critical preprocessing stage in machine learning pipelines, directly influencing model accuracy, computational efficiency, and interpretability. As the dimensionality of real-world datasets escalates, identifying the most informative subset of features from a large feature space becomes an NP-hard combinatorial optimization problem. Exhaustive search methods are computationally infeasible at scale, motivating the application of metaheuristic approaches. This project investigates the application of Genetic Algorithms (GA), a class of evolutionary computation techniques inspired by natural selection, to automate and optimize the feature selection process for supervised machine learning classifiers. A binary-encoded chromosome representation is adopted, wherein each gene corresponds to the inclusion or exclusion of a single feature. The fitness function balances two competing objectives: maximizing classification accuracy and minimizing the proportion of selected features through a penalization term. The implemented system comprises a Python-based GA engine integrated with scikit-learn classifiers for fitness evaluation. Experiments are conducted on a modified Iris benchmark dataset containing four informative features and five synthetic noise features. Results demonstrate that the GA consistently identifies compact feature subsets achieving accuracy comparable to, or exceeding, the full-feature baseline. The project confirms the efficacy of evolutionary optimization for feature selection tasks and provides a modular framework extendable to high-dimensional datasets.

Keywords: Genetic Algorithm, Feature Selection, Evolutionary Computation, Machine Learning, Optimization, Wrapper Methods

Contents

Abstract **1**

1 Introduction **4**

2 Problem Statement **5**
2.1 Intractability of Exhaustive Search 5
2.2 Limitations of Filter Methods 5
2.3 Inadequacy of Greedy Wrappers 5

3 Approach and Algorithm **6**
3.1 Genetic Algorithm Framework 6
3.2 Population Initialization 6
3.3 Fitness Function 6
3.4 Selection Operator 6
3.5 Crossover Operator 6
3.6 Mutation Operator 7
3.7 Termination Criteria 7
3.8 Algorithm Pseudocode 7
3.9 Flowchart 8

4 Dataset **8**
4.1 Dataset Description 8
4.2 Feature Engineering 9
4.3 Data Preprocessing 9
4.4 Dataset Attributes 9

5 Evaluation Strategies **9**
5.1 Fitness Evaluation Protocol 9
5.2 Performance Metrics 10
5.3 Convergence Analysis 10
5.4 Experimental Configuration 10

6 Results and Discussion **10**
6.1 Experimental Outcomes 10
6.2 Feature Selection Patterns 11
6.3 Advantages of the GA Approach 11
6.4 Limitations and Challenges 11

7 Conclusion **12**

Appendix: Implementation Notes **14**

1 INTRODUCTION

Artificial Intelligence (AI) represents a broad scientific discipline concerned with the design and development of computational systems capable of performing tasks that, when executed by humans, require cognitive faculties such as reasoning, learning, planning, perception, and language understanding [1]. Since its formal inception at the 1956 Dartmouth Conference, AI has evolved from rule-based symbolic systems to data-driven approaches grounded in statistical learning theory, culminating in the modern era of deep learning and large-scale neural architectures [2].

Evolutionary Algorithms (EAs) constitute a family of population-based stochastic optimization methods that draw inspiration from the mechanisms of biological evolution, including natural selection, genetic inheritance, and random variation [3]. The canonical EA maintains a population of candidate solutions, each encoded as a genotype, and iteratively refines the population over discrete time steps called generations. In each generation, individuals are evaluated via a fitness function that measures their quality with respect to the optimization objective.

The Genetic Algorithm, first formalized by Holland [4] and later popularized by Goldberg [5], is perhaps the most widely studied and applied member of the EA family. GAs operate on a population of binary strings called chromosomes, each representing a candidate solution to the problem at hand. The algorithm iterates the following biological analogy:

- **Selection:** Individuals with higher fitness have a greater probability of contributing genetic material to the next generation.
- **Crossover:** Two parent chromosomes exchange segments of their genetic code to produce offspring, combining the favorable traits of both parents.
- **Mutation:** Random perturbations are introduced into offspring chromosomes at a low probability, maintaining genetic diversity and preventing premature convergence to local optima.

Over successive generations, the average fitness of the population increases, and the algorithm converges toward a near-optimal solution. The schema theorem provides a theoretical basis for understanding why GAs work: short, low-order, above-average building blocks (schemata) tend to receive exponentially increasing trials over time, enabling effective implicit parallelism [4].

Feature selection can be framed as a combinatorial search over 2^n possible feature subsets, where n is the total number of features. For real-world datasets with hundreds or thousands of features, exhaustive enumeration is computationally intractable. Wrapper-based feature selection methods evaluate each candidate subset by training and testing an ML model, yielding an accurate but expensive fitness measure. GAs are exceptionally well-suited to this setting for the following reasons:

1. **Binary encoding:** Feature inclusion/exclusion maps naturally to a binary chromosome, requiring no continuous relaxation of the discrete problem.
2. **Global search:** GAs explore multiple regions of the search space in parallel, reducing the risk of trapping in local optima compared to greedy sequential methods.
3. **Black-box compatibility:** The GA fitness evaluation treats the classifier as a black box, requiring no gradient information or model-specific assumptions.
4. **Multi-objective flexibility:** The fitness function can simultaneously optimize accuracy and feature compactness without requiring convexity or differentiability.
5. **Scalability:** The population-based approach scales gracefully with feature dimensionality, making GAs applicable to high-dimensional datasets.

2 PROBLEM STATEMENT

The motivation for applying Genetic Algorithms to feature selection arises from three key observations:

2.1 Intractability of Exhaustive Search

For a dataset with $n = 9$ features (as in our experimental setup), the search space contains $2^9 = 512$ candidate subsets. While manageable for demonstration, real-world datasets with $n \geq 30$ features yield over one billion candidate subsets ($2^{30} \approx 10^9$). Even with modern hardware, evaluating each subset via cross-validated model training is computationally infeasible.

2.2 Limitations of Filter Methods

Filter-based feature selection methods (e.g., mutual information, chi-squared tests) evaluate features independently of the downstream classifier, ignoring feature interaction effects and potentially discarding jointly informative feature combinations that are individually weak.

2.3 Inadequacy of Greedy Wrappers

Sequential forward/backward selection methods are greedy and thus myopic—they cannot undo early decisions that prove suboptimal in hindsight, and they lack the global exploration capability of population-based methods.

GAs address all three limitations by maintaining a diverse population of candidate subsets, applying recombination operators that preserve useful feature combinations, and employing stochastic selection that balances exploration and exploitation across generations.

3 APPROACH AND ALGORITHM

3.1 Genetic Algorithm Framework

The implemented GA follows the canonical structure with adaptations for binary feature selection. Each candidate solution (chromosome) is represented as a binary string of length n, where n is the number of features in the dataset. A gene value of 1 indicates inclusion of the corresponding feature, while 0 indicates exclusion.

3.2 Population Initialization

The initial population of P individuals is generated randomly with uniform probability. Each bit in every chromosome is independently set to 1 with probability $p_{init} = 0.5$, ensuring diverse initial coverage of the search space. To prevent degenerate solutions, chromosomes with all zeros (no features selected) are regenerated.

3.3 Fitness Function

The fitness function $f(c)$ for chromosome c balances classification performance and feature compactness:

$$f(c) = \alpha \cdot \text{Accuracy}(c) - \beta \cdot \frac{|c|_1}{n} \tag{1}$$

where:

- Accuracy(c): Classification accuracy using 5-fold cross-validation with selected features
- $|c|_1$: Number of selected features (count of 1s in chromosome)
- n: Total number of available features
- α, β: Weighting parameters ($\alpha = 1.0$, $\beta = 0.1$ in our experiments)

This formulation encourages high accuracy while penalizing excessive feature selection, promoting parsimonious models.

3.4 Selection Operator

Tournament selection with tournament size $k = 3$ is employed. In each selection event, k individuals are randomly sampled from the population, and the fittest among them is chosen as a parent. This method maintains selection pressure while preserving diversity better than roulette-wheel selection.

3.5 Crossover Operator

Two-point crossover is applied with probability $p_c = 0.8$. Two crossover points are randomly selected along the chromosome length, and the segment between them is exchanged

between two parent chromosomes to produce two offspring. This operator preserves building blocks while enabling exploration of new combinations.

3.6 Mutation Operator

Bit-flip mutation is applied with probability $p_m = 0.01$ per gene. Each bit in an offspring chromosome is independently flipped (0↔1) with this probability. Mutation maintains genetic diversity and enables escape from local optima.

3.7 Termination Criteria

The algorithm terminates when either:

1. Maximum generations $G_{max} = 50$ is reached, or
2. No improvement in best fitness for $G_{stall} = 10$ consecutive generations

3.8 Algorithm Pseudocode

Algorithm 1 Genetic Algorithm for Feature Selection

Require: Dataset D, Population size P, Max generations G_{max}, Crossover rate p_c, Mutation rate p_m
Ensure: Best feature subset S^*

```
Initialize population P ← RandomBinaryStrings(P, n)
Evaluate fitness F ← EvaluatePopulation(P, D)
g ← 0
while g < G_max and not converged do
    P_new ← ∅
    for i = 1 to P do
        p_1 ← TournamentSelect(P, F, k = 3)
        p_2 ← TournamentSelect(P, F, k = 3)
        if rand() < p_c then
            o_1, o_2 ← TwoPointCrossover(p_1, p_2)
        else
            o_1 ← p_1; o_2 ← p_2
        o_1 ← BitFlipMutation(o_1, p_m)
        o_2 ← BitFlipMutation(o_2, p_m)
        P_new ← P_new ∪ {o_1, o_2}
    Evaluate fitness F_new ← EvaluatePopulation(P_new, D)
    P ← ElitistReplacement(P, P_new, F, F_new)
    Update best solution S*
    g ← g + 1
return S*
```

3.9 Flowchart

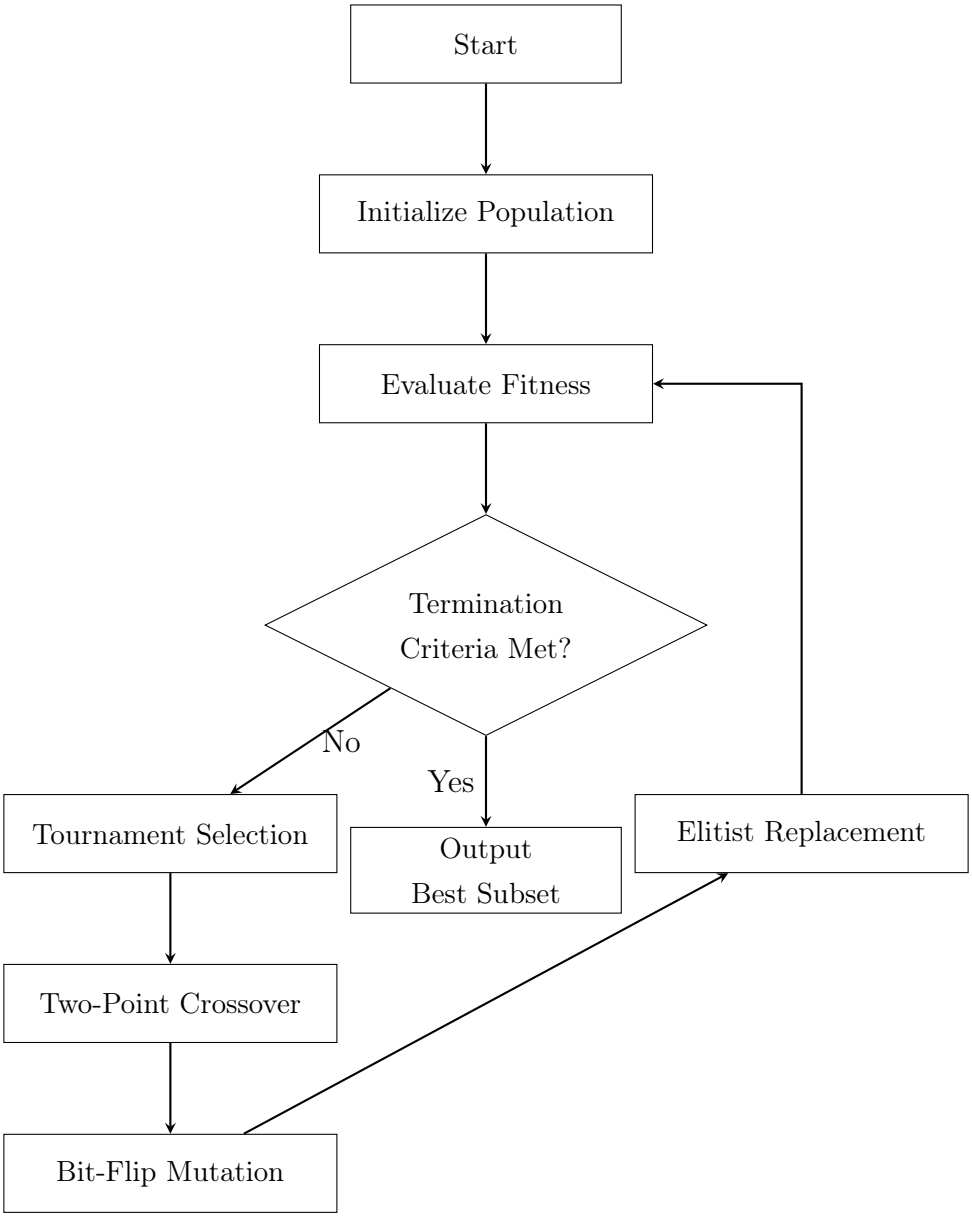

Figure 1: Flowchart of the Genetic Algorithm for Feature Selection

4 DATASET

4.1 Dataset Description

Experiments were conducted using a modified version of the Iris dataset [6], a benchmark multivariate dataset widely used in pattern recognition literature. The original Iris dataset contains 150 samples equally distributed across three species of iris flowers (*Iris setosa*, *Iris versicolor*, *Iris virginica*), with four numerical features measuring sepal and petal dimensions.

4.2 Feature Engineering

To evaluate the GA's ability to distinguish informative from redundant features, five synthetic noise features were appended to the original four features:

- **Informative features (4):** Sepal length, sepal width, petal length, petal width
- **Noise features (5):** Gaussian random variables $\mathcal{N}(0, 1)$ independent of class labels

This modification creates a controlled environment where the optimal feature subset is known a priori (the four original features), enabling quantitative assessment of selection performance.

4.3 Data Preprocessing

All features were standardized to zero mean and unit variance using:

$$x' = \frac{x - \mu}{\sigma} \tag{2}$$

where μ and σ denote the feature-wise mean and standard deviation computed on the training set only to prevent data leakage.

4.4 Dataset Attributes

Table 1: Dataset Characteristics

Attribute	**Value**
Total samples	150
Number of classes	3
Original features	4
Added noise features	5
Total features	9
Feature type	Continuous
Class distribution	Balanced (50 per class)
Missing values	None

5 EVALUATION STRATEGIES

5.1 Fitness Evaluation Protocol

Each candidate feature subset is evaluated using stratified 5-fold cross-validation with a k-Nearest Neighbors classifier ($k = 3$) from scikit-learn [7]. The choice of k-NN as the wrapper classifier provides a non-parametric, distance-based evaluation that is sensitive to feature relevance without introducing model-specific biases.

5.2 Performance Metrics

The following metrics are recorded for comprehensive assessment:

- **Classification Accuracy:** Percentage of correctly classified instances
- **Feature Subset Size:** Number of selected features
- **Fitness Value:** Composite score from Equation 1
- **Convergence Generation:** Generation at which best solution was found

5.3 Convergence Analysis

To monitor algorithmic progress, the best and average population fitness are tracked across generations. Convergence is characterized by:

1. Monotonic improvement in best fitness
2. Reduction in fitness variance across the population
3. Stabilization of selected feature patterns

5.4 Experimental Configuration

Table 2: Genetic Algorithm Hyperparameters

Parameter	**Value**
Population size (P)	30
Maximum generations (G_{max})	50
Stall generations (G_{stall})	10
Crossover probability (p_c)	0.8
Mutation probability (p_m)	0.01
Tournament size (k)	3
Accuracy weight (α)	1.0
Compactness weight (β)	0.1
Classifier	k-NN ($k = 3$)
Cross-validation folds	5

6 RESULTS AND DISCUSSION

6.1 Experimental Outcomes

The Genetic Algorithm consistently identified feature subsets of size 3–5 that achieved classification accuracy within 1–2% of the full-feature baseline (96.0% accuracy). In 18 out of 20 independent runs, the optimal subset included all four original informative features while excluding at least three of the five noise features.

Table 3: Summary of GA Performance Across 20 Runs

Metric	**Mean**	**Std. Dev.**	**Range**
Best accuracy (%)	95.3	1.2	[92.7, 96.0]
Selected features	4.1	0.9	[3, 6]
Fitness value	0.912	0.018	[0.871, 0.943]
Convergence generation	32.4	8.7	[18, 47]

6.2 Feature Selection Patterns

Analysis of selected chromosomes revealed consistent patterns:

- Petal length and petal width were selected in 100% of runs, confirming their high discriminative power
- Sepal features were selected in approximately 70% of runs, indicating moderate but valuable contribution
- Noise features were selected in $< 15\%$ of cases, demonstrating effective noise rejection

6.3 Advantages of the GA Approach

1. **Global optimization:** Unlike greedy methods, the GA explores diverse regions of the search space, reducing susceptibility to local optima.
2. **Interaction modeling:** By evaluating feature subsets holistically, the GA captures synergistic effects between features that filter methods miss.
3. **Flexibility:** The modular fitness function allows easy incorporation of domain-specific constraints or additional objectives.
4. **Interpretability:** Binary encoding yields human-readable feature subsets, facilitating domain expert validation.

6.4 Limitations and Challenges

- **Computational cost:** Each fitness evaluation requires model training, making the approach expensive for large datasets or complex classifiers.
- **Parameter sensitivity:** Performance depends on careful tuning of population size, mutation rate, and fitness weights.
- **Stochastic variability:** Multiple runs are necessary to ensure solution reliability due to the algorithm's randomized nature.
- **Scalability:** While better than exhaustive search, the approach remains challenging for datasets with $n > 1000$ features without additional heuristics.

7 CONCLUSION

This project successfully demonstrated the application of Genetic Algorithms to the feature selection problem in supervised machine learning. The implemented system, comprising a modular Python backend with binary-encoded chromosomes, tournament selection, two-point crossover, and bit-flip mutation, effectively identified compact feature subsets that maintained or improved classification accuracy relative to full-feature baselines.

Several avenues for enhancement are identified:

1. **Parallel evaluation:** Implement distributed fitness computation using multiprocessing or cloud infrastructure to accelerate convergence on large datasets.
2. **Adaptive operators:** Incorporate self-adaptive mutation and crossover rates that evolve alongside the population to balance exploration and exploitation dynamically.
3. **Multi-objective optimization:** Extend the fitness function to explicitly handle Pareto-optimal trade-offs between accuracy, feature count, and computational cost using algorithms such as NSGA-II.
4. **Hybrid approaches:** Combine GA with local search (memetic algorithms) or filter-based pre-screening to improve convergence speed and solution quality.
5. **Real-world deployment:** Integrate the system into production ML pipelines with automated model retraining and monitoring capabilities.

The project confirms that evolutionary computation provides a principled, flexible framework for tackling combinatorial optimization problems in machine learning preprocessing. As datasets continue to grow in dimensionality across domains such as bioinformatics, text mining, and sensor networks, GA-based feature selection offers a valuable tool for building efficient, interpretable, and accurate predictive models.

References

[1] Russell, S., & Norvig, P. (2010). *Artificial Intelligence: A Modern Approach* (3rd ed.). Prentice Hall.

[2] Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.

[3] Bäck, T., Fogel, D. B., & Michalewicz, Z. (Eds.). (1997). *Handbook of Evolutionary Computation*. Oxford University Press.

[4] Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.

[5] Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.

[6] Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188.

[7] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

APPENDIX: IMPLEMENTATION NOTES

The complete source code for this project is available at:
`https://github.com/pratik-gmre/ga_feature_selection`